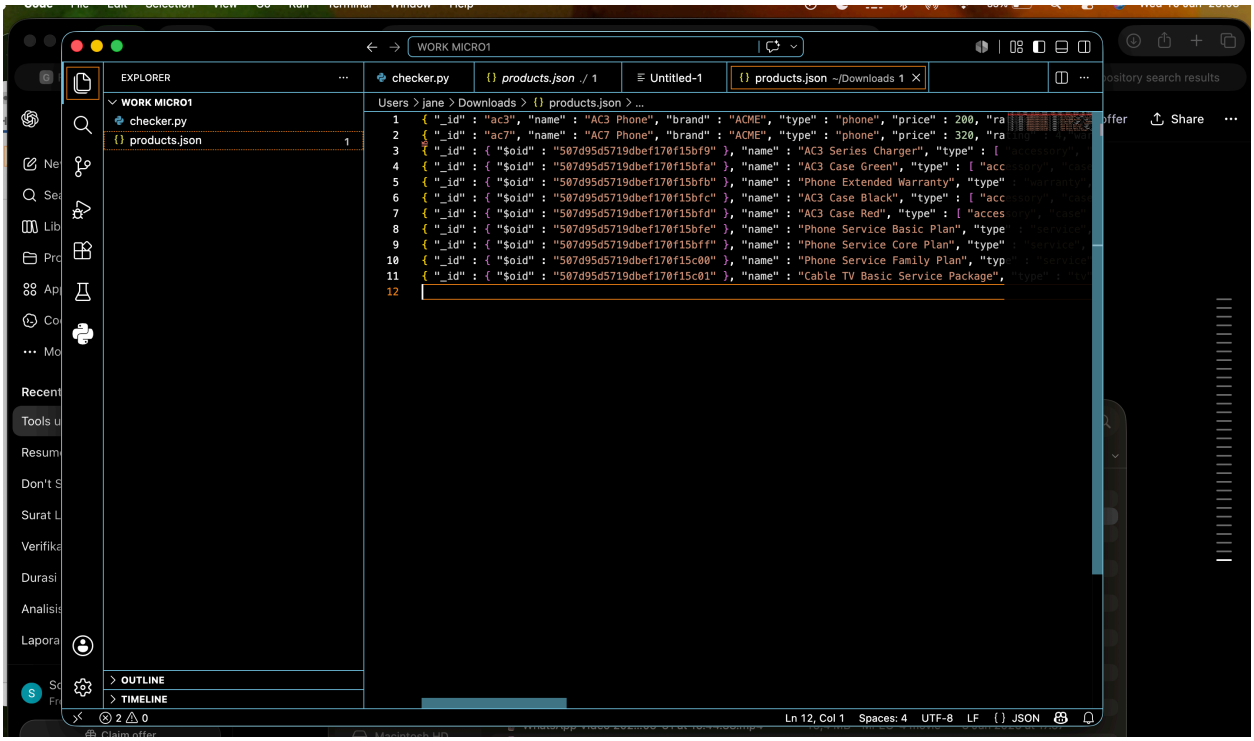
Example Explanation of My JSON Validation Process

Objective

The goal was to validate a product dataset stored in JSON format and identify data quality issues before performing manual review. Since manually reviewing every record can be time-consuming and error-prone, I used a simple Python validation script to automate repetitive checks and quickly identify potential problems.

Step 1: Understand the Dataset Structure and formatting the data



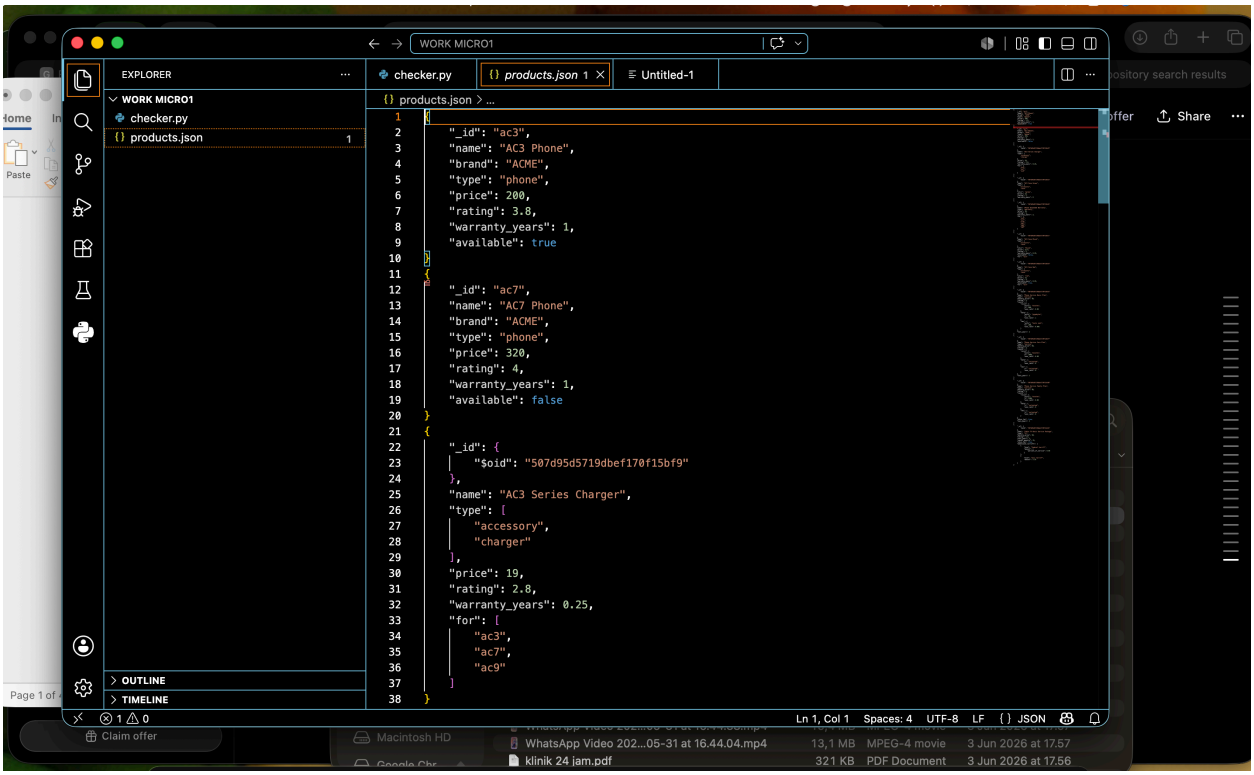


Image after Formatting JSON

Before writing any code with python for automation, I reviewed the product data to understand its structure and identify common fields such as:

- _id
- name
- type
- rating
- available
- for

The objective was to determine what a valid record should look like and identify areas where inconsistencies might occur. Its for understanding the schema is necessary before creating validation rules.

Step 2: Load and Parse the JSON Data

I created a Python script to read the product file and convert the records into Python objects. The data needed to be loaded into memory so it could be analyzed automatically instead of reviewing each product manually.

The objectibe is to Transform raw JSON records into a format that could be validated programmatically.

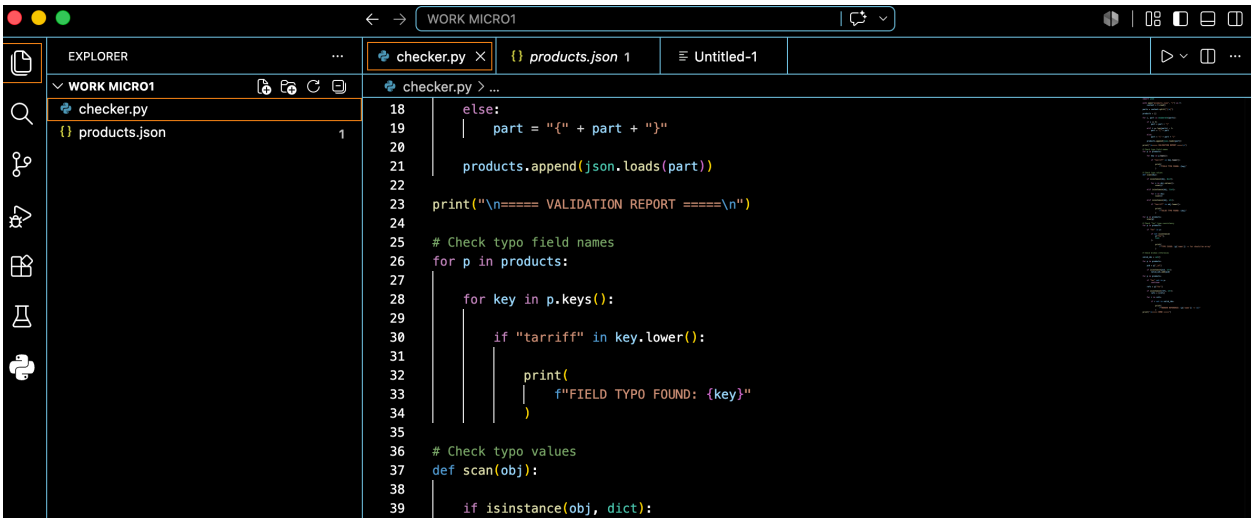
Step 3: Verify That the Data Can Be Read Successfully

The first validation was simply confirming that all product records could be loaded correctly. If the file cannot be parsed correctly, all subsequent validation checks become unreliable. Its to ensure the dataset is readable and structurally usable.

Step 4: Check for Field Name Errors

I added a validation rule to search for suspicious field names, specifically looking for misspellings such as:

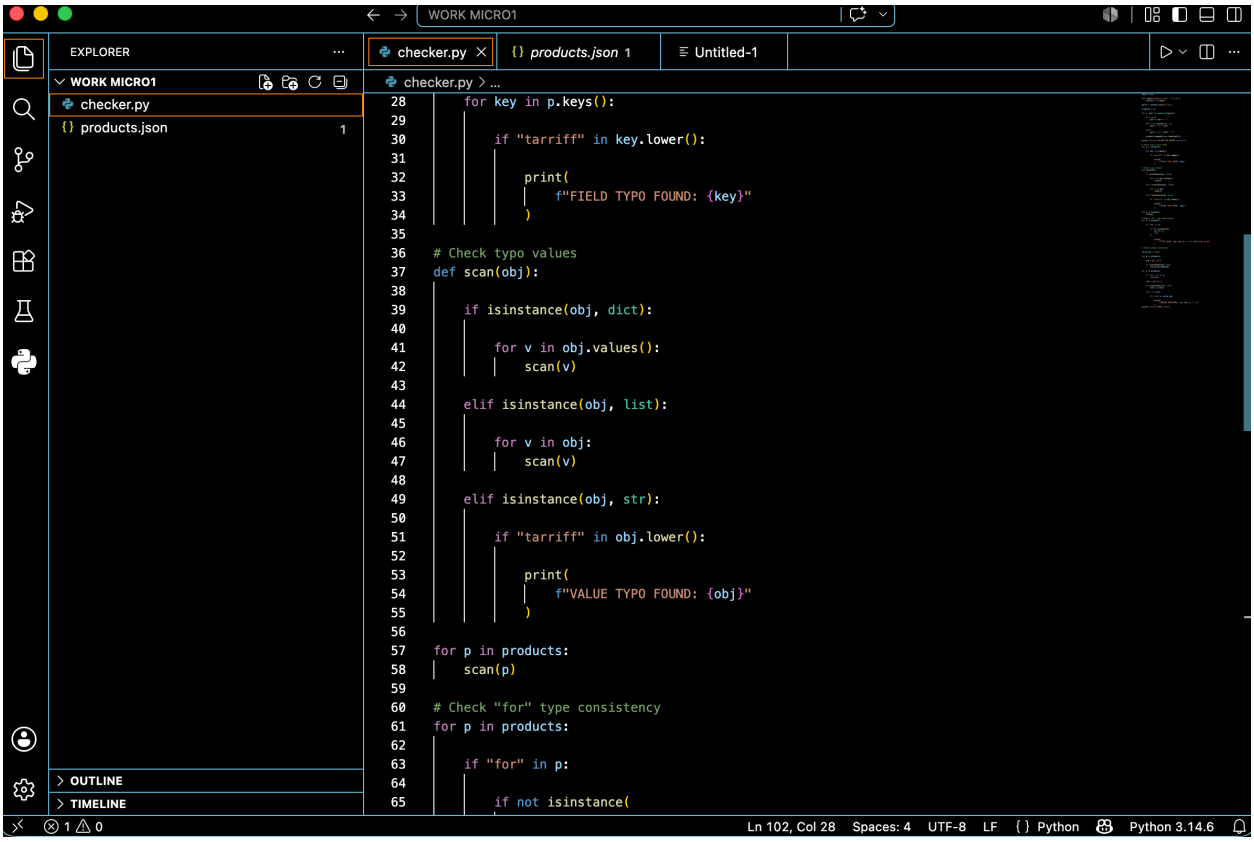
additional_tarriffs
Misspelled field names can break downstream systems because applications expect exact field names.



The script identified a typo in the field name. Ensure schema consistency across all records.

Step 5: Check for Incorrect Text Values

I created a recursive scan that searched every string value in the dataset.Typographical errors can exist not only in field names but also inside the values themselves.

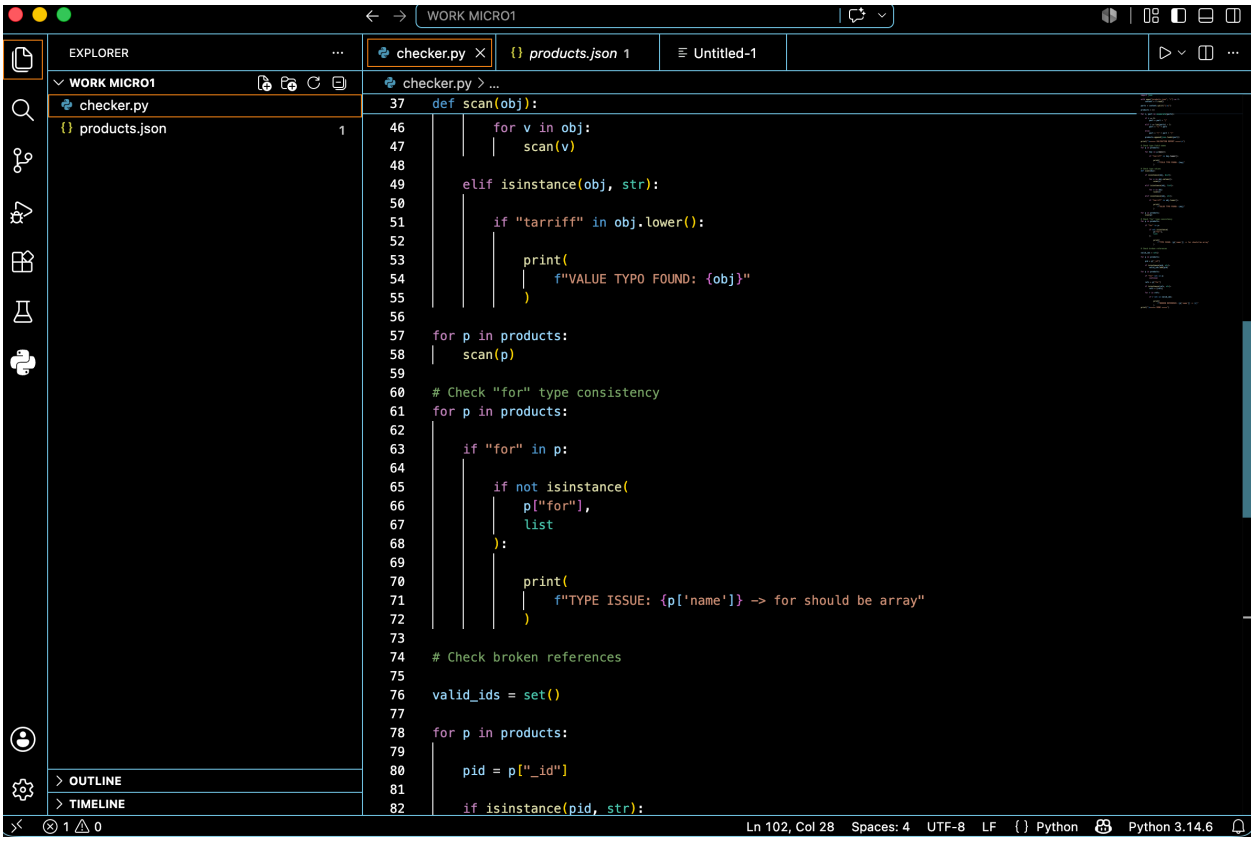



```python
    for key in p.keys():

        if "tarriff" in key.lower():

            print(
                f"FIELD TYPO FOUND: {key}"
            )

# Check typo values
def scan(obj):

    if isinstance(obj, dict):

        for v in obj.values():
            scan(v)

    elif isinstance(obj, list):

        for v in obj:
            scan(v)

    elif isinstance(obj, str):

        if "tarriff" in obj.lower():

            print(
                f"VALUE TYPO FOUND: {obj}"
            )

for p in products:
    scan(p)

# Check "for" type consistency
for p in products:

    if "for" in p:

        if not isinstance(
```

Findings:

- federal tarriff
- misc tarriff

Step 6: Check Data Type Consistency




```python
def scan(obj):
        for v in obj:
            scan(v)

    elif isinstance(obj, str):

        if "tarriff" in obj.lower():

            print(
                f"VALUE TYPO FOUND: {obj}"
            )

for p in products:
    scan(p)

# Check "for" type consistency
for p in products:

    if "for" in p:

        if not isinstance(
            p["for"],
            list
        ):

            print(
                f"TYPE ISSUE: {p['name']} -> for should be array"
            )

# Check broken references

valid_ids = set()

for p in products:

    pid = p["_id"]

    if isinstance(pid, str):
```

I validated the "for" field.

Expected structure:

["ac3"]

Observed structure:

"ac3"

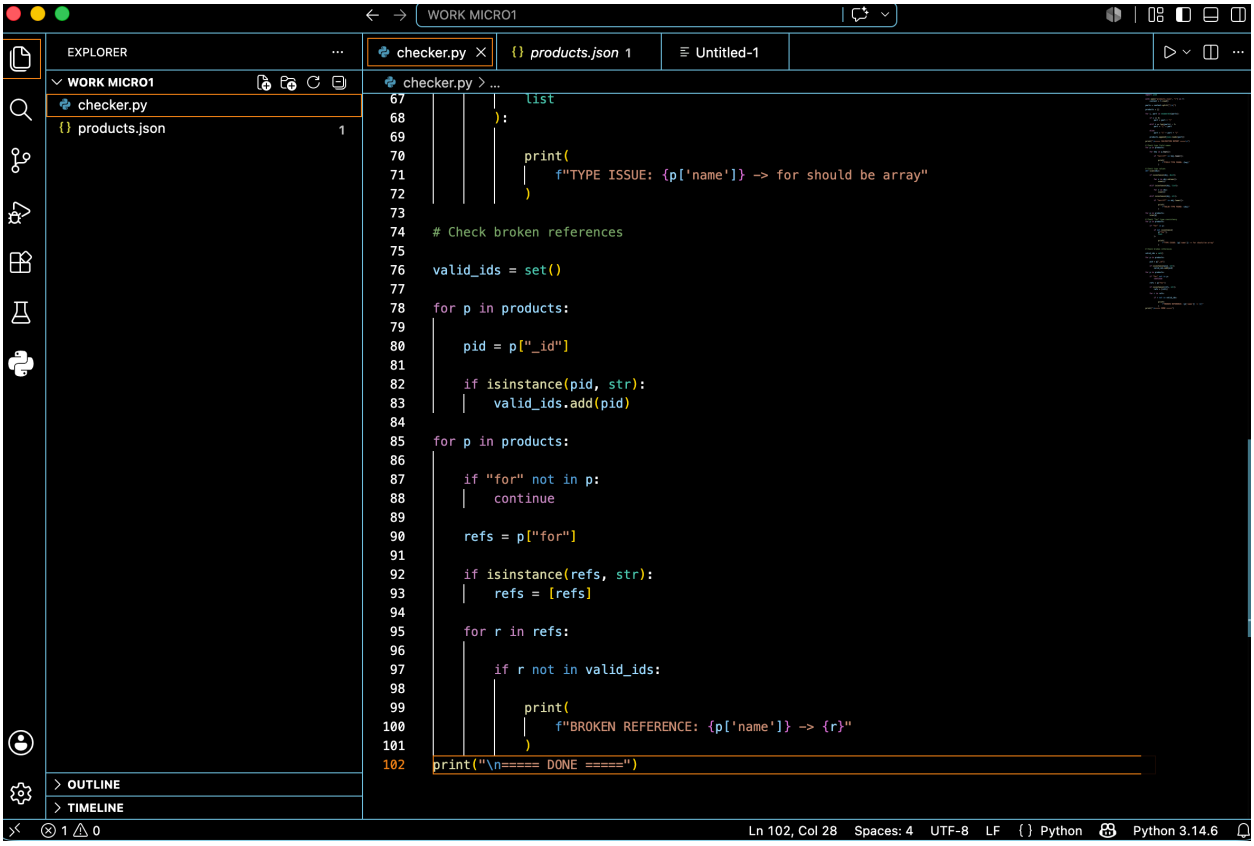
A field should maintain a consistent data type across all records. Mixed types can cause application errors and validation failures.

Findings:

- AC3 Case Black
- AC3 Case Red

Step 7: Check Reference Integrity

I collected all valid product IDs and compared them against references stored inside the "for" field. A reference should point to an existing product.



Findings:

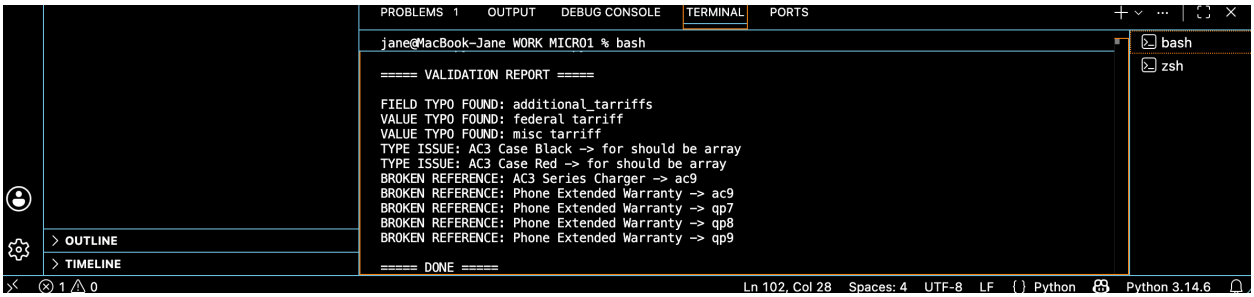
- ac9
- qp7
- qp8
- qp9

These IDs were referenced but did not exist in the available dataset.

Step 8: Generate a Validation Report

The script produced a structured report containing all identified issues. A validation report allows reviewers to focus only on problematic records instead of inspecting every record manually.

.



The automated validation process identified:

- Field name typo
- Value typos
- Data type inconsistencies
- Broken references

How I validate this JSON dataset?

First, I will review the JSON structure to understand the schema and expected fields. Then I perform automated validation checks to identify potential issues such as field name inconsistencies, typographical errors, data type mismatches, and broken references. After reviewing the automated findings, I manually verify the flagged records and document all issues in a structured validation report.

I identified several data quality issues: A misspelled field name (additional_tarriffs),Misspelled values (federal tarriff, misc tarriff),Inconsistent data types in the for field, where some records used strings and others used arrays, and Broken references to product IDs that were not present in the dataset (ac9, qp7, qp8, qp9).These issues could cause downstream validation, mapping, or processing errors.

I use automation because manually reviewing large datasets is time-consuming and increases the risk of overlooking errors. Automated validation allows me to quickly identify structural issues and focus manual review efforts on more complex data quality and document-level checks.